ExamCraft

Online Examination Management System

Software Requirements Specification

Version 1.0
Prepared for academic project documentation
20 September 2026

Document status	Baseline SRS
System type	Web-based online examination platform
Primary users	Institute Administrator, Department Administrator/Teacher, Student
Technology baseline	React, Vite, Tailwind CSS, Node.js, Express, MySQL, JWT, bcrypt, Joi

Document Control

Version	Date	Description
1.0	20 Sep 2026	Initial consolidated SRS based on the ExamCraft project repository, use cases, DFD, ER model, and implemented routes.

Table of Contents

• 1. Introduction

• 2. Overall Description

• 3. User Roles and Permissions

• 4. Functional Requirements

• 5. External Interface Requirements

• 6. Non-Functional Requirements

• 7. Data Requirements

• 8. System Architecture

• 9. Use Cases

• 10. Acceptance Criteria

• 11. Assumptions and Constraints

• 12. Future Enhancements

• Appendix A: API and Route Summary

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional, non-functional, data, interface, security, and operational requirements for ExamCraft, a web-based system for creating, scheduling, conducting, monitoring, and reporting online examinations. It is intended for students, institute administrators, department administrators or teachers, developers, testers, project evaluators, and future maintainers.

1.2 Product Scope

ExamCraft provides a role-based examination workflow: an institute administrator establishes the institute structure; department administrators manage students and author exams; and students take assigned exams through a timed interface. The system supports question banks, MCQ, descriptive, and coding questions, answer persistence, monitoring events, automatic or forced submission, result processing, reports, audit records, and institute settings.

1.3 Definitions and Acronyms

Term	Meaning
SRS	Software Requirements Specification
JWT	JSON Web Token used for authenticated sessions
Institute Admin	Administrator who manages an institute and its departments
Department Admin	Teacher or department-level administrator
Attempt	A student’s execution of an assigned exam
Question Bank	Reusable collection of exam questions
SEB	Safe Exam Browser or equivalent controlled exam environment

2. Overall Description

2.1 Product Perspective

ExamCraft is a client-server single-page application. The React frontend communicates with a Node.js/Express REST backend. The backend authenticates users, applies role and institute/department scope, validates requests, persists data in MySQL, and manages exam lifecycle operations. Docker Compose can run the frontend, backend, and MySQL services together.

2.2 Product Functions

• Institute registration and institute administrator access.

• Department creation and department administrator management.

• Student registration, listing, and account access control.

• Exam authoring, scheduling, publishing, editing, and assignment.

• Question bank management with reusable question groups.

• Timed student exam delivery with navigation, answer saving, heartbeat, and submission.

• Support for MCQ, descriptive, and coding question workflows.

• Live monitoring of assigned students and active attempts.

• Force submission, automatic timeout submission, basic scoring, and reports.

• Audit logging, notifications, file metadata, settings, and activity tracking.

2.3 Operating Environment

Layer	Environment
Client	Modern desktop browser with JavaScript enabled; responsive React interface.
Frontend	React, Vite, React Router HashRouter, Tailwind CSS.
Backend	Node.js 22 or compatible LTS runtime, Express.js.
Database	MySQL accessed through mysql2; local Docker or configured remote MySQL.
Deployment	Docker Compose; frontend port 5173 and backend port 3000 by default.

2.4 Design and Implementation Constraints

• The system must preserve institute and department data isolation.

• Passwords must not be stored in plaintext; authentication uses bcrypt password hashes.

• Protected API operations require an authenticated cookie-based JWT session.

• Exam content contains flexible JSON fields for options, answers, sections, and coding test cases.

• The current implementation uses browser-side restrictions as deterrence; these controls cannot guarantee prevention of all cheating on an unmanaged device.

• Coding execution must be isolated and resource-limited before production deployment.

2.5 Assumptions

• Each user belongs to one institute and may belong to one department.

• Students are assigned to an exam before they can start it.

• An exam has a start time, duration, status, and question content.

• The database schema is initialized before normal application use.

• Administrative users are trusted to configure valid exam content and student assignments.

3. User Roles and Permissions

Role	Core permissions	Scope
Institute Administrator	Register/manage institute, create departments, create department admins, view dashboards/reports, review audit activity, configure settings.	Own institute
Department Administrator / Teacher	Manage students, freeze/unfreeze access, create/edit exams, manage question banks, assign students, monitor live attempts, force-submit attempts, generate department reports.	Own department and institute
Student	Log in, view assigned exams, read instructions, start/resume, answer, save, submit, view status/results, submit coding solutions.	Own account and assignments

4. Functional Requirements

4.1 Authentication and Authorization

FR-AUTH-01 — The system shall provide a unified login endpoint requiring email, password, and role.

FR-AUTH-02 — The system shall support institute, department, and student roles.

FR-AUTH-03 — The system shall reject invalid credentials, inactive accounts, invalid roles, and role mismatches.

FR-AUTH-04 — The system shall hash passwords using bcrypt before storage and compare hashes during login.

FR-AUTH-05 — The system shall issue an HttpOnly JWT cookie after successful authentication.

FR-AUTH-06 — The system shall verify token validity, expiration, role, and device fingerprint on protected requests.

FR-AUTH-07 — The system shall update the user's last-login timestamp after successful login.

FR-AUTH-08 — The system shall return appropriate authentication errors without exposing password or hash data.

4.2 Institute and Department Administration

FR-ORG-01 — The system shall allow creation of an institute with name, code, address, contact email, and phone.

FR-ORG-02 — The system shall allow an institute administrator to create departments with name, code, and optional head email.

FR-ORG-03 — The system shall prevent duplicate department codes within the same institute.

FR-ORG-04 — The system shall allow creation of department administrator accounts associated with a valid institute and department.

FR-ORG-05 — The system shall restrict institute administrators to their own institute.

FR-ORG-06 — The system shall display institute-level counts and activity information where available.

FR-ORG-07 — The system shall support institute-level CSV reporting for departments, users, and exams where implemented.

FR-ORG-08 — The system shall record significant administrative actions in audit logs.

4.3 Student Management

FR-STU-01 — The system shall allow department administrators to register students with identity, contact, institute, department, role, and semester information.

FR-STU-02 — The system shall list students belonging to the authenticated administrator's department.

FR-STU-03 — The system shall allow department administrators to activate or deactivate student access.

FR-STU-04 — The system shall reject student operations outside the administrator's department.

FR-STU-05 — The system shall record student access changes in the audit log.

4.4 Exam Authoring and Scheduling

FR-EXAM-01 — The system shall allow authorized department users to create an exam with title, description, duration, total marks, start date/time, department, and institute.

FR-EXAM-02 — The system shall support draft, scheduled, active, completed, and other configured exam states.

FR-EXAM-03 — The system shall validate that duration is a positive whole number and that an optional end time is after the start time.

FR-EXAM-04 — The system shall support online and offline exam modes as configured.

FR-EXAM-05 — The system shall support exam metadata including subject, domain, question count, passing marks, sections, and question-bank information.

FR-EXAM-06 — The system shall allow authorized users to edit exam details subject to attempt-history constraints.

FR-EXAM-07 — The system shall assign selected students to an exam and preserve assignments linked to historical attempts.

FR-EXAM-08 — The system shall prevent students from accessing exams outside their institute, assignment, or permitted exam window.

4.5 Question Banks and Questions

FR-Q-01 — The system shall support reusable question-bank records and saved question-bank groups.

FR-Q-02 — The system shall support MCQ, descriptive, and coding question types.

FR-Q-03 — The system shall preserve question order, section labels, marks, options, correct answers, and answer mode.

FR-Q-04 — The system shall permit exam questions to be authored directly or sourced from a reusable bank.

FR-Q-05 — The system shall support coding question details including language, starter code, visible/hidden test cases, time limit, and memory limit.

FR-Q-06 — The system shall validate question payloads before persisting them.

4.6 Student Exam Delivery

FR-DEL-01 — The system shall display available and in-progress assigned exams to the student.

FR-DEL-02 — The system shall display exam instructions before the attempt begins.

FR-DEL-03 — The system shall create or resume an attempt for an eligible student.

FR-DEL-04 — The system shall display questions with navigation and progress state.

FR-DEL-05 — The system shall autosave selected answers and provide save confirmation or error feedback.

FR-DEL-06 — The system shall maintain a countdown timer based on the configured duration and attempt state.

FR-DEL-07 — The system shall send heartbeat/activity information while an attempt is active.

FR-DEL-08 — The system shall automatically submit an attempt when the timer expires or configured conditions require it.

FR-DEL-09 — The system shall allow final submission and prevent further answer modification after submission.

FR-DEL-10 — The system shall support marked-for-review and time-spent information where enabled.

4.7 Evaluation and Coding Execution

FR-EVAL-01 — The system shall calculate objective marks from submitted answers according to question marks and answer rules.

FR-EVAL-02 — The system shall store total score, total marks, submission status, and completion time.

FR-EVAL-03 — The system shall accept coding submissions associated with an attempt and question.

FR-EVAL-04 — The coding runner shall return execution status, output, errors, runtime, and marks where implemented.

FR-EVAL-05 — The production coding runner shall isolate submitted code and enforce time, memory, process, and filesystem restrictions.

4.8 Monitoring, Proctoring, and Reports

FR-MON-01 — The system shall provide department users with live exam and student-attempt status.

FR-MON-02 — The system shall show enrolled, started, submitted, auto-submitted, and not-started counts.

FR-MON-03 — The system shall record configured activity events such as tab switches, fullscreen exits, idle warnings, copy/cut/paste attempts, and devtools attempts.

FR-MON-04 — The system shall allow an authorized department user to force-submit an active attempt.

FR-MON-05 — The system shall record force submission and other monitoring actions in audit logs.

FR-MON-06 — The system shall generate department exam reports in CSV format where implemented.

FR-MON-07 — The system shall show basic exam results and attempt status to authorized users and students according to role.

4.9 Supporting Services

FR-SUP-01 — The system shall support user notifications for relevant exam events where enabled.

FR-SUP-02 — The system shall store uploaded-file metadata for supported profile, question, or document files.

FR-SUP-03 — The system shall support institute settings using a unique institute/key combination.

FR-SUP-04 — The system shall maintain audit records containing actor, action, table, record, values, and timestamp where available.

5. External Interface Requirements

5.1 User Interface

• The interface shall provide separate login and protected dashboard experiences for institute, department, and student users.

• The student exam workspace shall show the current question, navigation controls, progress, timer, answer state, and submit action.

• Administrative dashboards shall show summary cards, lists, quick actions, exam controls, student management, and monitoring information.

• Validation messages shall identify the affected field or operation and shall not expose internal stack traces.

• The interface shall be usable at common desktop resolutions and provide clear visual status for draft, active, submitted, and error states.

5.2 Software Interfaces

Interface	Requirement
Frontend–Backend	React client shall communicate with Express REST endpoints using JSON and credentialed requests.
Authentication	Protected requests shall carry the HttpOnly token cookie; clients shall not need direct access to the token value.
Database	Backend shall use mysql2 connections/pooling and parameterized SQL statements.
Configuration	Environment variables shall provide port, database, JWT, CORS, and optional TLS configuration.
Reports	Report endpoints shall return downloadable CSV with suitable content type and filename.

5.3 Communication Interfaces

• HTTP/HTTPS shall be used between browser and backend.

• CORS shall explicitly allow configured frontend origins and credentials.

• Production deployment shall use HTTPS, secure cookies, and a properly managed JWT secret.

• Database TLS shall be enabled when connecting to a remote MySQL provider, using a trusted CA certificate.

6. Non-Functional Requirements

NFR-01 Security — Passwords shall be bcrypt-hashed; SQL shall be parameterized; authorization shall check role and institute/department ownership.

NFR-02 Confidentiality — Question correct answers, password hashes, JWT secrets, and internal errors shall not be exposed to unauthorized clients.

NFR-03 Availability — The application shall return a controlled error response when database or dependent services are unavailable and shall not crash the whole process for a single bad request.

NFR-04 Performance — Normal dashboard and list requests should complete within 3 seconds under the expected academic-project load, excluding network and database outages.

NFR-05 Scalability — The backend shall use connection pooling and support horizontal deployment when session and file storage are externalized or consistently shared.

NFR-06 Reliability — Answer saves and final submissions shall be idempotent or safely repeatable so network retries do not duplicate answers or attempts.

NFR-07 Usability — The system shall provide understandable labels, validation messages, confirmations for destructive actions, and visible timer/submission states.

NFR-08 Maintainability — Frontend modules shall be organized by role/function; backend routes, validation schemas, middleware, and database access shall remain separable.

NFR-09 Compatibility — The system shall support current Chromium-, Firefox-, and Edge-based desktop browsers with JavaScript enabled.

NFR-10 Auditability — Administrative and exam lifecycle events shall retain actor, action, target record, and timestamp where applicable.

NFR-11 Data Integrity — Foreign-key relationships, unique constraints, transactions, and server-side scope checks shall protect organization and attempt data.

NFR-12 Accessibility — Controls shall have meaningful labels, visible focus states, readable contrast, and keyboard-accessible navigation where practical.

7. Data Requirements

7.1 Main Entities

Entity	Important attributes	Purpose
institutes	institute_id, name, code, address, contact_email, phone	Top-level organization
departments	department_id, institute_id, name, code, head_email	Academic/administrative unit
users	user_id, institute_id, department_id, email, password_hash, role, name, phone, status	Identity and access
subjects	subject_id, department_id, code, name, credits, syllabus	Academic subject information
exams	exam_id, scope, title, description, schedule, duration, marks, status, mode	Exam definition
questions	question_id, exam_id, type, text, options, answer, marks, order	Exam content
question_bank / groups	bank/group IDs, metadata, JSON questions, source exam	Reusable content
exam_assignments	assignment_id, exam_id, student_id	Exam–student mapping
exam_attempts	attempt_id, assignment, student, timestamps, score, status, timer	Attempt lifecycle
student_answers	answer_id, attempt, question, selected answer, marks, review state	Saved/evaluated answers
activity/audit logs	attempt/user, action, values, timestamp, client data	Monitoring and traceability

7.2 Integrity Rules

• Institute codes and user emails shall be unique where defined by the schema.

• A department shall belong to one institute; users shall not be assigned across institute boundaries.

• An exam assignment shall reference an existing exam and student.

• An answer shall reference an existing attempt and question, with one logical answer state per attempt/question where enforced.

• Deleting or modifying records with attempts shall preserve required historical records or be rejected according to referential-integrity rules.

• JSON fields shall contain valid JSON and the backend shall normalize arrays before persistence.

8. System Architecture

8.1 Logical Architecture

Component	Responsibilities
React SPA	Routing, role-specific dashboards, forms, exam workspace, timer display, client-side interaction controls, API calls.
Express API	Routing, request validation, authentication middleware, authorization, exam lifecycle, reports, monitoring, and business rules.
Authentication subsystem	bcrypt password verification, JWT creation/verification, HttpOnly cookie, fingerprint check, session expiry.
MySQL database	Persistent organization, user, exam, question, assignment, attempt, answer, monitoring, and audit data.
Code runner	Compilation/execution and result comparison for coding submissions; must be sandboxed for production.
Docker environment	Repeatable local deployment of frontend, backend, and database services.

8.2 Main Data Flow

Institute registration → authentication → department/user setup → exam authoring → student assignment → exam instructions/start → answer saving and monitoring → submission/evaluation → results and reports.

8.3 Security Architecture

Every protected route shall pass authentication middleware. Role checks shall be applied before privileged operations. Department and institute IDs shall be resolved from the authenticated user whenever possible, rather than trusted solely from request bodies. Database operations shall use parameterized queries. Sensitive configuration shall be supplied through environment variables and excluded from source control.

9. Use Cases

ID	Use case	Actor	Description
UC-01	Authenticate User	All users	User submits valid credentials and selected role; system validates account, creates session cookie, and redirects to the role dashboard. Invalid or inactive accounts receive an error.
UC-02	Manage Institute	Institute Admin	Administrator registers institute or views institute information; system validates uniqueness and persists the organization.
UC-03	Manage Departments	Institute Admin	Administrator creates a department and optionally identifies its head; system verifies institute scope and records the action.
UC-04	Manage Students	Department Admin	Administrator registers, lists, activates, or deactivates students in the department.
UC-05	Create and Assign Exam	Department Admin	Teacher defines exam metadata, questions, schedule, security settings, and student assignment; system validates and saves the exam.
UC-06	Take Exam	Student	Student opens an assigned exam, reads instructions, starts/resumes an attempt, answers questions, saves progress, and submits.
UC-07	Submit Coding Solution	Student	Student submits source code for a coding question; system executes it under limits and stores status/output/marks.
UC-08	Monitor Exam	Department Admin	Teacher opens live monitoring to see student states and activity, then may force-submit an active attempt.
UC-09	Generate Report	Institute Admin / Department Admin	Authorized user requests a report; system filters by scope and returns dashboard data or CSV.

10. Acceptance Criteria

• A valid user can log in with the correct role and is redirected to the corresponding protected dashboard.

• An invalid role, inactive account, wrong password, expired token, or fingerprint mismatch is rejected.

• An institute administrator can create a department only inside the authenticated institute.

• A department administrator can manage only students and exams within the assigned department.

• A teacher can create an exam with valid schedule, duration, marks, questions, security options, and student assignments.

• An assigned student can start or resume an exam and see a working countdown timer.

• Answer changes survive refresh or a retry after autosave succeeds.

• An exam submits normally through the submit action and automatically when time expires.

• Live monitoring displays meaningful student/attempt states and authorized force submission changes the attempt status.

• Reports contain correct scope and downloadable CSV formatting.

• The system records required audit events and does not expose passwords, hashes, JWT secrets, or correct answers to students.

• The application can be started using the documented Node/Docker setup with a configured MySQL database.

11. Assumptions and Constraints

• The supplied repository is the authoritative baseline; some advanced modules are represented in schema/design documents and may require completion or hardening before production use.

• The browser is not a trusted security boundary. Copy/paste, tab-switch, fullscreen, and devtools detection should be treated as monitoring signals, not absolute prevention.

• Email/SMS delivery, enterprise SSO, biometric proctoring, and production-grade remote code execution are outside the current baseline scope.

• The current backend contains runtime schema checks/ALTER statements for backward compatibility; production deployments should use versioned database migrations instead.

• The application requires correct timezone handling between browser-local input, backend UTC conversion, and database timestamps.

12. Future Enhancements

• Detailed analytics and item/question performance reports.

• Email/SMS/push notifications and calendar integration.

• Enterprise SSO and multi-factor authentication.

• Versioned question banks with approval workflow and import/export.

• Accessible rich-text/descriptive answer evaluation and rubric grading.

• Production sandboxed code execution using isolated containers or a dedicated judge service.

• Stronger proctoring integrations with privacy notices, consent, retention controls, and human review.

• Automated database migrations, automated tests, CI/CD, observability, rate limiting, and backup/restore procedures.

Appendix A: API and Route Summary

Route group	Representative operations	Primary role
/api/auth	Login, verify, institute-admin register, department-admin register, student register.	All
/api/institute	Institute setup and department administration.	Institute Admin
/api/institute/dashboard	Institute dashboard, activity, settings, reports.	Institute Admin
/api/department	Department creation and related operations.	Institute Admin
/api/department/dashboard	Department dashboard, students, live exams, monitoring, force submit, reports.	Department Admin
/api/exams	Create/list/view/update exams, assignments, attempts, answers, submission, status.	Department/Student
/api/question-banks	List and save reusable question banks/groups.	Department Admin
/api/student/dashboard	Student dashboard and available exam information.	Student
/api/run	Coding submission execution and evaluation.	Student
/api/user	User profile or account data operations.	Authenticated users

Appendix B: Implementation Notes

• The frontend uses HashRouter paths such as /login, /exams, /exam/:examId, /institute-dashboard, and /department-dashboard.

• The backend listens on port 3000 by default; Vite development frontend uses port 5173 by default.

• The repository includes DFD and ER documentation that should be kept synchronized with schema migrations and route behavior.

• This SRS describes intended system behavior and should be updated when a requirement, role permission, database relationship, or API contract changes.